Workshop · Module 03

Smarter features, *honest* benchmark.

03 / 08

Agenda

ARC 1 — THE RICHER MODEL

Every column. Every table.

01 **Categoricals and dates — passed as-is**
gender, marital status, education, occupation, secondary_income_flag, account_open_date. NEXUS reads them directly — cleaning and joins add signal.

02 **Three left joins, one feature frame**
Financial snapshots. Credit assessments. Payment events. Merged on borrower_id.

03 **Messy data — clean lightly**
Case drift, type drift, typos. NEXUS tolerates them. Clean for hygiene, not because you have to.

ARC 2 — THE HONEST FIGHT

NEXUS vs XGBoost, same frame

01 **Same data, same holdout, same metric**
The rules of a fair fight — stated out loud before any training begins.

02 **~30 lines of XGBoost preprocessing**
OrdinalEncoder, SimpleImputer, ColumnTransformer. Shown in full — nothing hidden.

03 **Exercise — overlay the ROCs**
Three curves on one chart. Pick the model you would ship — and say why.

By the end of this module, you'll be able to…

Three things you'll *leave with*

A

Enrich a feature frame with *mixed types*

Pass categoricals and dates to NEXUS as-is — no encoder, no date-to-number step. Then add cleaning and joins that lift accuracy. NEXUS handles NaNs natively and tolerates light mess in the source.

B

Join three tables onto one borrower record

Sort by date, drop duplicates, left merge. Financial snapshots, credit assessments, and payment aggregates land as new columns — and move the AUC.

C

Benchmark NEXUS against *tuned XGBoost*

Same data, same split, same metric. Compare AUC, preprocessing lines, and tuning time — then read the ROC overlay for the full picture.

PART 01

From four *columns*, to every *column*.

The one-sentence definition.

Strings, dates, nulls, *three joined tables* — passed to one fit() call as-is.

FIG. 03 · THE MODULE IN ONE SENTENCE

WHY IT WORKS

Mixed types, *handled natively*.

NEXUS handles mixed-type tables natively. You pass categorical strings and date columns as-is — no encoder, no date-to-number step — and it reads them directly. It already understands how features interact and how a missing value differs from a zero.

NEXUS adapts to your data on the fit() call. You hand it the raw frame, and it represents the columns for you — the work you would otherwise spend encoding and imputing is simply not on your plate.

WHAT THAT MEANS

Pass the frame as-is, and NEXUS adapts to your data

No ColumnTransformer. No imputation pipeline. No encoder for the categoricals, no transform for the date — NEXUS reads the raw columns and adapts to them on fit().

as-is not *encode · impute · grid-search.*

CATEGORICALS AND DATES

Passed *as-is.*

NEXUS takes categoricals and dates as-is. You pass *Single*, *Married*, and *Bachelor's* straight through — no encoder — and hand `account_open_date` over as a date. Cleaning and joins add signal: less feature engineering, not zero.

For the borrower table that means five new feature types join the four numerics from Module 01 — no encoder, no ColumnTransformer.

NEW IN MODULE 03

Five new feature types, passed as-is

gender, marital_status, education_level, occupation_sector — plus account_open_date, read directly as a temporal feature.

---

no encoder categoricals and dates read directly — no pipeline.

Three tables. *Three signals.*

Each join adds a column set NEXUS trains on — a different dimension of risk the others cannot cover.

01

Financial snapshots

monthly_income, income_growth, collateral_score, secondary_income_flag. Latest snapshot per borrower — what their finances look like *now*.

02

Credit *assessments*

Six analyst-scored dimensions of creditworthiness. How underwriters *judge* the borrower today — a distilled expert signal.

03

Payment *events*

total_payments, on_time_rate, avg_payment_usd. Summary stats, not rolling windows — NEXUS learns the pattern from the aggregates.

enrich() — three joins, one frame.

Sort by date. Drop duplicates. Left merge. Standard pandas, applied straight.

module_03_smarter_features.ipynb · cell 23

```python
# latest snapshot per borrower, latest assessment, aggregated payments
# — each prepared upstream; here we just join them on.

def enrich(df):
    out = df.copy()
    out = out.merge(snapshots_latest,   on="borrower_id", how="left")
    out = out.merge(assessments_latest, on="borrower_id", how="left")
    out = out.merge(payment_agg,        on="borrower_id", how="left")
    return out

train_enriched   = enrich(train_raw)
holdout_enriched = enrich(holdout_raw)

# missing values from failed joins stay NaN — NEXUS handles them natively.
# auto-imputation would bake in assumptions we have not checked.
```

WHY THIS MATTERS

Three joins, zero pipelines.

Sort by date. `drop_duplicates(keep="first")`. `merge(..., how="left")`. Idiomatic pandas, nothing bespoke. NEXUS trains on the result directly — no `ColumnTransformer`, no fit/transform split, no inference-time preprocessing to reproduce.

THE TEST

Messy data? *NEXUS still trains.*

We fed the enriched frame to NEXUS *without* cleanup. Case drift (`DIVORCED` vs `Divorced`), typos (`Maale`, `Femle`), floats where ints belong. No pipeline, no fix-up.

AUC on the messy frame passed as-is: **0.8937**. The model still learns — but case variants and typos eat into the signal.

A light cleanup first — `.str.title()`, typo fixes, int casts — not imputation, not binning — lifts AUC to **0.9164** (+0.0227). NEXUS reads `Divorced` and `DIVORCED` as two categories; consistent casing consolidates them into one.

THE HABIT

Clean lightly, not because the model breaks

Standardize casing, fix obvious typos, cast dtypes. Skip imputation, binning, and transformations unless a domain reason says otherwise. You clean for *hygiene*, not because the pipeline demands it.

+0.02 AUC from a two-line cleanup — not from deep imputation.

Now the *benchmark*.
Same data. Same split. Same metric.

What XGBoost *needs first*.

Every line NEXUS does not.

module_03_smarter_features.ipynb · cell 40

```
from sklearn.preprocessing import OrdinalEncoder
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
# column types must be identified by hand
cat_cols = ["gender", "marital_status", "education_level",
            "occupation_sector", "secondary_income_flag"]
num_cols = [c for c in features
            if c not in cat_cols + ["account_open_date"]]
cat_pipe = Pipeline([
    ("impute", SimpleImputer(strategy="most_frequent")),
    ("encode", OrdinalEncoder(handle_unknown="use_encoded_value",
                              unknown_value=-1)),
])
preprocessor = ColumnTransformer([
    ("num", SimpleImputer(strategy="median"), num_cols),
    ("cat", cat_pipe, cat_cols),
])  # account_open_date is silently excluded
```

WHY THIS MATTERS

Shape the frame, then train.

Before XGBoost ever sees a row: identify numeric and categorical columns by hand, impute strategies that are implicit domain choices, ordinal-encode strings with an unknown-value guard. None of this goes away at inference — it has to be reproduced on every new request.

THE GAP

The date column *silently drops*.

`account_open_date` is an ISO string. `ColumnTransformer` does not know what to do with it, so the preprocessor on the previous slide discards it by omission — no error, no warning.

XGBoost trains on a frame that is one column short. The temporal signal in account tenure never reaches the model.

THE POINT

What's not in the comparison matters too

NEXUS reads ISO dates as temporal signals — tenure, seasonality — with no code change on your side. XGBoost *could* be made to see the date, but only by adding parse/extract/fillna logic that then has to be reproduced at inference.

1 feature dropped from the XGBoost pipeline without a warning.

THE NUMBERS

Three models. *One honest table.*

XGBoost (defaults) is the baseline — ~30 lines of preprocessing, no tuning, typical AUC around **0.90**. XGBoost (tuned) adds a GridSearchCV pass — **~0.91**, a small bump for meaningful wall-clock cost. NEXUS (quality) lands around **0.94** on the raw cleaned frame, no encoding and no tuning.

On raw, cleaned data NEXUS leads. Give XGBoost the same feature engineering and the gap narrows — NEXUS's edge is largest before the feature work. AUC is one dimension; preprocessing lines, tuning time, and features silently excluded belong in the same sentence as the number. The next slide shows the curves.

WHAT AUC MISSES

The code you did not write counts too

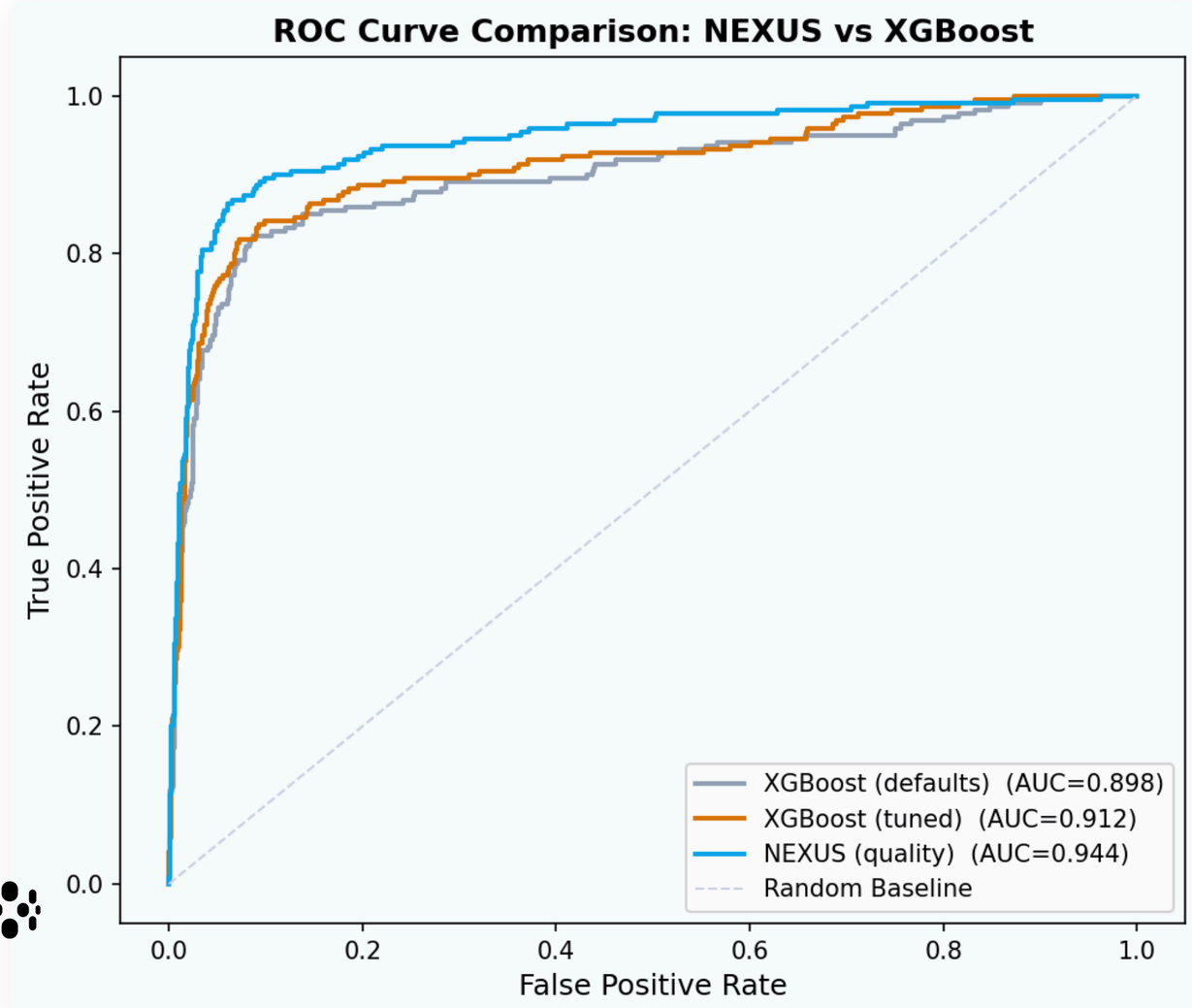
XGBoost preprocessing breaks on new category values at inference. Grid search scales badly with dataset size. The date column is lost. NEXUS keeps the date as a feature, needs no grid search, and carries a far smaller preprocessing surface.

+0.03 AUC

NEXUS over tuned XGBoost on raw data — gap narrows with equal feature engineering.

The *overlay*.

Three curves, one holdout — the shape of the gap, not just the number.



READING THE OVERLAY

The gap is consistent across the whole threshold range

NEXUS sits above XGBoost at nearly every operating point — not just the one the AUC summary captures. That matters in credit risk, where you often operate at low false-positive rates and the curve there decides ship-or-not.

schematic exact numbers come from your notebook run.

EXERCISE

03

Overlay the ROCs. Defend your pick.

FORMAT	Solo, at your laptop
MATERIALS	Enriched train and holdout frames
OUTPUT	Three-curve ROC + one-sentence verdict
DIFFICULTY	Intermediate · ●●●○

PARTICIPANT INSTRUCTIONS

Train three candidates on the same enriched frame. Pick the model *you would ship* — and say why.

01 Run the XGBoost *defaults* cell. Note AUC and wall-clock time.

02 Run the *GridSearchCV* cell. While it runs, count the preprocessing lines NEXUS did not need.

03 Generate the overlaid ROC chart. Compare the three curves visually — not just by AUC.

04 In one sentence: which model ships, and what you traded to pick it. Paste into the notebook's *Your call* cell.

Three *takeaways*

If you forget everything else, remember these.

01

Mixed types, *passed as-is*

NEXUS takes categoricals and dates as-is — no encoder, no ColumnTransformer, no imputation. Cleaning and joins are additive: less feature engineering, not zero.

02

Joins *move the AUC*

Financial snapshots, credit assessments, payment aggregates. Three left merges on borrower_id — each adds a column worth training on.

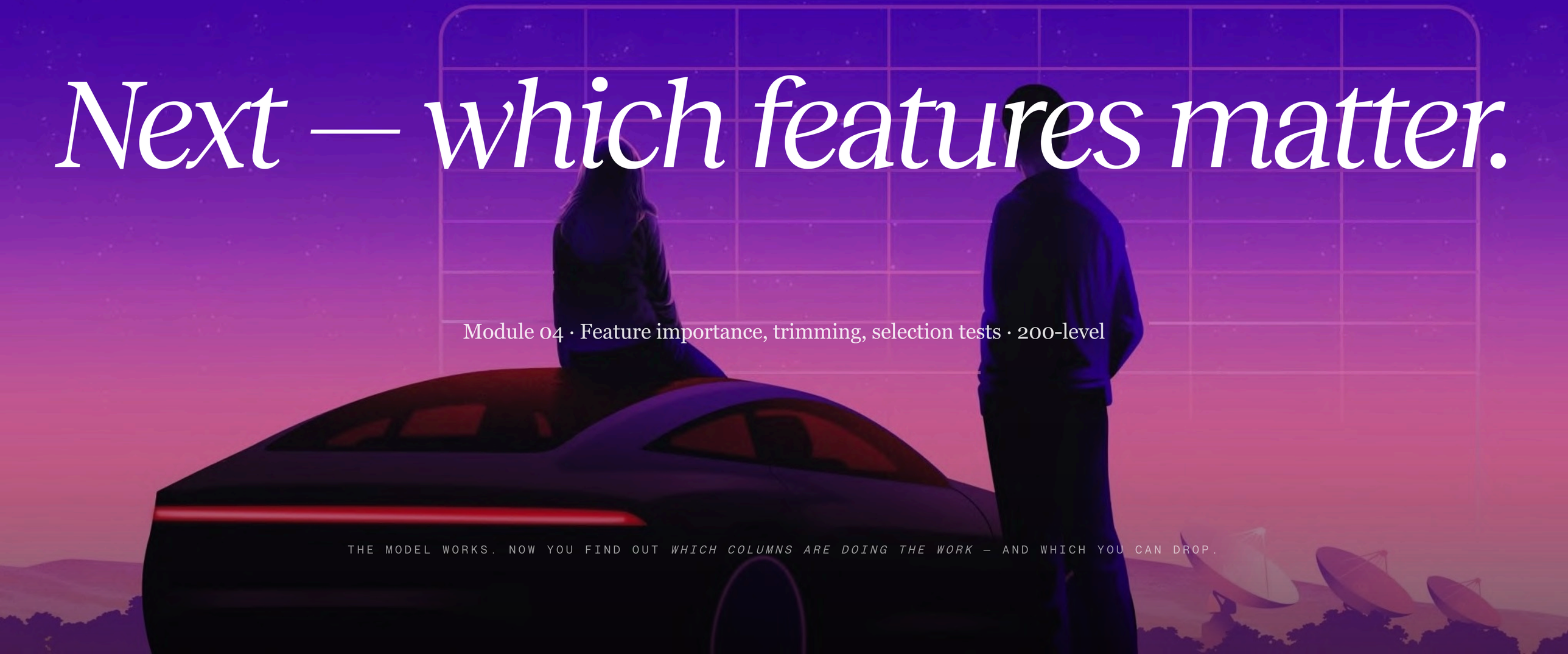
03

The benchmark is *honest*

AUC is one dimension. Preprocessing lines, tuning time, and features silently excluded belong in the same comparison.

NEXUS onboarding | Fundamental

Next — which features matter.



Module 04 · Feature importance, trimming, selection tests · 200-level

THE MODEL WORKS. NOW YOU FIND OUT *WHICH COLUMNS ARE DOING THE WORK* — AND WHICH YOU CAN DROP.